Fact API Backend Implementation: Routing & User Input Processing

Comprehensive Technical Specification & Architecture Design

Module: Core Fact API Service

Version: 2.1.0

Status: Technical Blueprint

Date: October 2026

1. Architecture Overview & System Design

1.1 Purpose: The Fact API Backend service is designed to process incoming client queries, sanitize user input, manage intelligent endpoint routing, and aggregate structured factual responses with minimal latency.

1.2 Core Stack: Built on Node.js / Express (or Python FastAPI), leveraging asynchronous event loops, middleware validation pipelines, and modular router controllers to maintain separation of concerns.

1.3 High-Level Flow: Client Request -> API Gateway -> Security Middleware -> Input Sanitizer -> Dynamic Router -> Fact Controller -> Data Store / External Provider -> Response Serializer.

2. Project Directory Structure

2.1 Modular Layout: Maintaining clean folder organization for scalability and ease of maintenance across microservices.

```
/fact-api-backend
│
├── /src
│   ├── /config       # Database and environment configurations
│   ├── /controllers  # Request handlers and business logic
│   ├── /middlewares  # Authentication, validation, and error logging
│   ├── /routes       # Modular API route definitions
│   ├── /services     # Core fact processing & external API clients
│   └── app.ts        # Express/FastAPI application entry point
├── /tests            # Unit and integration test suites
├── package.json
└── tsconfig.json
```

3. Application Initialization & Server Setup

3.1 Server Entry Point (`app.ts`): Initializing the server framework, configuring global middleware like CORS, rate-limiting, and body parsers.

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import factRouter from './routes/fact.routes';
import { errorHandler } from './middlewares/error.middleware';

const app = express();

app.use(helmet());
app.use(cors());
app.use(express.json({ limit: '10kb' }));

// Mount Fact API Routes
app.use('/api/v1/facts', factRouter);

// Global Error Handling Middleware
app.use(errorHandler);

export default app;
```

3.2 Port Binding & Lifecycle: Managing graceful shutdowns and environment variable loading.

4. Routing Architecture & Endpoint Design

4.1 Router Configuration (`fact.routes.ts`): Defining explicit endpoints for fetching, submitting, and validating facts.

```
import { Router } from 'express';
import {
    getFactsController,
    createFactController,
    searchFactsController
} from '../controllers/fact.controller';
import { validateUserInput } from '../middlewares/validation.middleware';
import { authMiddleware } from '../middlewares/auth.middleware';

const router = Router();

router.get('/', getFactsController);
router.get('/search', searchFactsController);
router.post('/', authMiddleware, validateUserInput, createFactController);

export default router;
```

5. User Input Validation & Sanitization

5.1 Security Requirements: Preventing injection attacks, XSS, and malformed payloads by implementing strict schema validation using libraries like Zod or Joi.

5.2 Validation Middleware (`validation.middleware.ts`):

```
import { Request, Response, NextFunction } from 'express';
import { z } from 'zod';

const factSchema = z.object({
    title: z.string().min(3).max(150),
    category: z.enum(['science', 'history', 'technology', 'geography']),
    content: z.string().min(10).max(2000),
    tags: z.array(z.string()).optional()
});

export const validateUserInput = (req: Request, res: Response, next: NextFunction) => {
    try {
        const validatedData = factSchema.parse(req.body);
        req.body = validatedData; // Sanitized payload
        next();
    } catch (error) {
        return res.status(400).json({
            success: false,
            error: 'Validation failed',
            details: error instanceof z.ZodError ? error.errors : 'Invalid input format'
        });
    }
};
```

6. Fact Controller & Business Logic Layer

6.1 Controller Implementation (`fact.controller.ts`): Handling business workflows and interacting with underlying services.

```
import { Request, Response, NextFunction } from 'express';
import { FactService } from '../services/fact.service';

export const getFactsController = async (req: Request, res: Response, next: NextFunction) => {
    try {
        const { category, limit = '10', page = '1' } = req.query;
        const result = await FactService.fetchFacts({
            category: category as string,
            limit: parseInt(limit as string, 10),
            page: parseInt(page as string, 10)
        });
        return res.status(200).json({ success: true, data: result });
    } catch (error) {
        next(error);
    }
};
```

7. Advanced Search & Query Processing

7.1 Search Controller Extension: Implementing full-text matching and query parameter filtering.

```
export const searchFactsController = async (req: Request, res: Response, next: NextFunction) => {
    try {
        const query = req.query.q as string;
        if (!query || query.trim().length === 0) {
            return res.status(400).json({ success: false, error: 'Query parameter "q" is required
        }
        const searchResults = await FactService.searchQuery(query.trim());
        return res.status(200).json({
            success: true,
            count: searchResults.length,
            data: searchResults
        });
    } catch (error) {
        next(error);
    }
};
```

7.2 Indexing Strategy: Utilizing database indexes on text search fields to ensure queries execute in under 50ms.

8. Error Handling & Exception Management

8.1 Global Error Middleware (`error.middleware.ts`): Centralized error formatting and stack trace suppression in production environments.

```
import { Request, Response, NextFunction } from 'express';

export const errorHandler = (err: any, req: Request, res: Response, next: NextFunction) => {
    const statusCode = err.statusCode || 500;
    const message = err.message || 'Internal Server Error';

    console.error(`[Error] ${new Date().toISOString()} - ${message}`, err.stack);

    return res.status(statusCode).json({
        success: false,
        status: statusCode,
        error: message,
        ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
    });
};
```

9. Authentication & Security Middleware

9.1 JWT Verification (`auth.middleware.ts`): Securing mutation endpoints (POST, PUT, DELETE) against unauthorized access.

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';

export interface AuthenticatedRequest extends Request {
    user?: any;
}

export const authMiddleware = (req: AuthenticatedRequest, res: Response, next: NextFunction) => {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
        return res.status(401).json({ success: false, error: 'Access token missing or malformed' });
    }

    const token = authHeader.split(' ')[1];
    try {
        const secret = process.env.JWT_SECRET || 'default_secret';
        const decoded = jwt.verify(token, secret);
        req.user = decoded;
        next();
    } catch (error) {
        return res.status(403).json({ success: false, error: 'Invalid or expired token' });
    }
};
```

10. Testing, Quality Assurance & Deployment

10.1 Unit Testing with Jest & Supertest: Ensuring route handlers and validation layers perform as expected under simulated client traffic.

```
import request from 'supertest';
import app from '../src/app';

describe('Fact API Endpoints', () => {
    it('should retrieve a list of facts successfully', async () => {
        const response = await request(app).get('/api/v1/facts');
        expect(response.status).toBe(200);
        expect(response.body.success).toBe(true);
        expect(Array.isArray(response.body.data)).toBe(true);
    });

    it('should reject invalid fact payloads', async () => {
        const response = await request(app)
            .post('/api/v1/facts')
            .send({ title: 'Ab' }); // Fails min length validation
        expect(response.status).toBe(400);
        expect(response.body.success).toBe(false);
    });
});
```

10.2 Deployment Checklist: Environment variable configuration, Docker containerization, health check monitoring (`/health`), and reverse proxy setup via Nginx.